

ML-DL-Ops Assignment 5

Name: Arpita Abhijit Deshmukh Roll No: B23CM1007

Objective

This assignment is divided into two parts. Q1 involves fine-tuning a pre-trained ViT-Small model on CIFAR-100 — first without LoRA, then with LoRA using PEFT for various rank and alpha combinations, followed by hyperparameter optimization using Optuna. Q2 focuses on adversarial attacks using IBM Adversarial Robustness Toolbox (ART) on CIFAR-10 with ResNet-34, covering FGSM (from scratch and via ART) and adversarial detection using PGD and BIM attacks. All experiments were conducted inside a Docker container.

Q1: ViT-S Fine-tuning on CIFAR-100 with LoRA

Setup & Model

- **Base Model:** ViT-Small (ViT-S/16) pre-trained on ImageNet via `timm`
- **Dataset:** CIFAR-100 (50,000 train / 10,000 test, 100 classes)
- **Classification Head:** Replaced with a linear layer (`hidden_dim` → 100)
- **Training:** 10 epochs, AdamW optimizer, CosineAnnealingLR scheduler
- **LoRA via PEFT:** Injected into Q, K, V attention projection weights
- **LoRA Dropout:** Fixed at 0.1 for all experiments

Experiment 0 (No LoRA): Fine-tune Classification Head Only

Epoch	Train Loss	Val Loss	Train Acc (%)	Val Acc (%)
1	1.1430	0.7910	70.61	77.08
2	0.6761	0.7334	80.29	78.47
3	0.5899	0.7356	82.66	78.49
4	0.5364	0.7144	84.13	79.15
5	0.4936	0.7196	85.34	78.95
6	0.4604	0.7122	86.28	78.88
7	0.4352	0.7108	87.07	79.45
8	0.4142	0.7090	87.84	79.54
9	0.3987	0.7059	88.38	79.39
10	0.3902	0.7045	88.75	79.48

Experiment 0 — No LoRA, classification head only

LoRA Experiments: Train-Val Tables per Configuration

The following table template is repeated for all 9 rank–alpha combinations ($\text{Rank} \in \{2, 4, 8\}$, $\text{Alpha} \in \{2, 4, 8\}$, Dropout = 0.1):

Experiment 1 — Rank: 2, Alpha: 2, Dropout: 0.1

Epoch	Train Loss	Val Loss	Train Acc (%)	Val Acc (%)
1	0.6810	0.3821	82.00	87.98
2	0.2907	0.3591	90.89	88.92
3	0.2186	0.3606	92.97	89.18
4	0.1682	0.3537	94.59	89.57
5	0.1315	0.3673	95.95	89.38
6	0.1043	0.3647	96.90	89.69
7	0.0840	0.3654	97.75	89.62
8	0.0704	0.3697	98.28	89.68
9	0.0620	0.3710	98.65	89.76
10	0.0578	0.3712	98.83	89.78

Experiment 2 — Rank: 2, Alpha: 4, Dropout: 0.1

Epoch	Train Loss	Val Loss	Train Acc (%)	Val Acc (%)
1	0.6799	0.3716	82.05	88.67
2	0.2859	0.3568	91.07	88.98
3	0.2147	0.3640	93.12	88.96
4	0.1637	0.3540	94.79	89.34
5	0.1267	0.3620	95.99	89.57
6	0.0968	0.3643	97.17	89.59
7	0.0761	0.3641	98.04	89.76
8	0.0629	0.3679	98.51	89.55
9	0.0539	0.3692	98.92	89.64
10	0.0491	0.3692	99.08	89.67

Experiment 3 — Rank: 2, Alpha: 8, Dropout: 0.1

Epoch	Train Loss	Val Loss	Train Acc (%)	Val Acc (%)
1	0.6529	0.3908	82.63	87.77
2	0.2847	0.3645	90.93	88.81
3	0.2130	0.3664	93.08	89.12
4	0.1607	0.3662	94.83	89.55
5	0.1208	0.3723	96.22	89.38
6	0.0904	0.3799	97.44	89.29
7	0.0691	0.3777	98.14	89.55
8	0.0552	0.3802	98.78	89.70
9	0.0465	0.3821	99.08	89.69
10	0.0416	0.3830	99.27	89.70

Experiment 4 — Rank: 4, Alpha: 2, Dropout: 0.1

Epoch	Train Loss	Val Loss	Train Acc (%)	Val Acc (%)
1	0.6847	0.3869	82.06	88.02
2	0.2871	0.3464	90.86	89.30
3	0.2127	0.3569	93.10	89.21
4	0.1642	0.3496	94.74	90.04
5	0.1235	0.3523	96.14	90.02
6	0.0951	0.3588	97.20	89.93
7	0.0743	0.3619	98.06	90.27
8	0.0623	0.3644	98.57	90.22
9	0.0534	0.3665	98.89	90.17
10	0.0501	0.3665	99.03	90.13

Experiment 5 — Rank: 4, Alpha: 4, Dropout: 0.1

Epoch	Train Loss	Val Loss	Train Acc (%)	Val Acc (%)
1	0.6658	0.3742	82.48	88.56
2	0.2823	0.3570	90.93	89.12
3	0.2040	0.3570	93.29	89.07
4	0.1519	0.3541	95.09	89.86
5	0.1110	0.3649	96.48	89.67
6	0.0816	0.3745	97.66	89.54
7	0.0608	0.3716	98.49	89.70
8	0.0480	0.3788	99.02	89.86
9	0.0407	0.3783	99.29	90.00
10	0.0367	0.3793	99.40	89.90

Experiment 6 — Rank: 4, Alpha: 8, Dropout: 0.1

Epoch	Train Loss	Val Loss	Train Acc (%)	Val Acc (%)
1	0.6552	0.3767	82.61	88.40
2	0.2750	0.3480	91.19	89.32
3	0.1941	0.3639	93.77	89.30
4	0.1385	0.3671	95.61	89.41
5	0.0977	0.3759	97.04	89.70
6	0.0691	0.3787	98.11	89.88
7	0.0494	0.3824	98.81	89.99
8	0.0367	0.3867	99.33	90.00
9	0.0302	0.3872	99.52	90.11
10	0.0269	0.3877	99.66	90.10

Experiment 7 — Rank: 8, Alpha: 2, Dropout: 0.1

Epoch	Train Loss	Val Loss	Train Acc (%)	Val Acc (%)
1	0.6850	0.3734	81.95	88.38
2	0.2844	0.3578	91.14	89.10
3	0.2106	0.3400	93.21	89.84
4	0.1575	0.3577	94.94	89.42
5	0.1189	0.3595	96.35	89.69
6	0.0896	0.3665	97.43	89.56
7	0.0687	0.3685	98.27	89.68
8	0.0560	0.3726	98.73	89.70
9	0.0479	0.3724	99.04	89.82
10	0.0447	0.3726	99.17	89.71

Experiment 8 — Rank: 8, Alpha: 4, Dropout: 0.1

Epoch	Train Loss	Val Loss	Train Acc (%)	Val Acc (%)
1	0.6755	0.3708	82.12	88.41
2	0.2748	0.3463	91.29	89.15
3	0.1977	0.3603	93.67	89.18
4	0.1409	0.3537	95.54	89.73
5	0.0986	0.3603	97.04	89.76
6	0.0707	0.3692	98.12	89.98
7	0.0516	0.3761	98.81	89.79
8	0.0396	0.3789	99.26	89.73
9	0.0331	0.3805	99.45	89.78
10	0.0299	0.3807	99.57	89.96

Experiment 9 — Rank: 8, Alpha: 8, Dropout: 0.1

Epoch	Train Loss	Val Loss	Train Acc (%)	Val Acc (%)
1	0.6556	0.3721	82.49	88.79
2	0.2693	0.3479	91.44	89.52
3	0.1844	0.3574	94.13	89.79
4	0.1267	0.3657	96.00	89.89
5	0.0846	0.3725	97.48	90.06
6	0.0553	0.3883	98.56	90.01
7	0.0370	0.3898	99.20	90.04
8	0.0365	0.3893	99.55	90.25
9	0.0210	0.3926	99.71	90.20
10	0.0182	0.3993	99.78	90.24

Test Accuracy Summary Table

LoRA Layers	Rank	Alpha	Dropout	Overall Test Acc (%)	Trainable Params
Without LoRA (head only)	–	–	–	79.54	38500
Q, K, V	2	2	0.1	89.78	75364
Q, K, V	2	4	0.1	89.76	75364
Q, K, V	2	8	0.1	89.70	75364
Q, K, V	4	2	0.1	90.27	112228
Q, K, V	4	4	0.1	90.00	112228
Q, K, V	4	8	0.1	90.11	112228
Q, K, V	8	2	0.1	89.84	185956
Q, K, V	8	4	0.1	89.98	185956
Q, K, V	8	8	0.1	90.25	185956

Test accuracy comparison across all LoRA configurations and baseline

Optuna Hyperparameter Search

- **Tool:** Optuna with TPESampler
- **Objective:** Maximize validation accuracy
- **Search Space:** Rank $\in \{2, 4, 8\}$, Alpha $\in \{2, 4, 8\}$, Dropout fixed at 0.1
- **Number of Trials:** 9
- **Best Configuration Found:** Rank = 4, Alpha = 2, Dropout = 0.1
- **Best Validation Accuracy:** 90.27 %

Key Observations

- Higher alpha/rank ratio generally leads to stronger adaptation but risks overfitting on limited epochs.
- LoRA significantly increased the number of trainable parameters while keeping pre-trained backbone weights frozen, offering a strong accuracy-efficiency trade-off.
- Optuna converged to an optimal LoRA configuration in faster than exhaustive grid search.

Q2: Adversarial Attacks using IBM ART on CIFAR-10

ResNet-18 Baseline Training (Clean CIFAR-10)

Epoch	Val Acc (%)
1	33.21
2	48.84
3	57.22
4	60.46
5	67.10
6	72.69

ResNet-18 training on CIFAR-10 ($\geq 72\%$ test accuracy target (Early stopping))

FGSM Attack — From Scratch vs IBM ART

FGSM Attack Results — Accuracy vs Perturbation Strength (ϵ)

ϵ	Adversarial Acc — Scratch (%)	Adversarial Acc — ART (%)
0.01	58.86	67.31
0.05	22.16	35.77
0.10	8.55	21.94

FGSM at various perturbation strengths ϵ

Key Observations — FGSM

- Both scratch and ART-based FGSM implementations produced visually similar adversarial images.
- Small perturbations were produced which were imperceptible to the human eye yet highly effective at fooling the model.
- Model accuracy drops steeply with increasing ϵ , with near-random performance ($\sim 10\%$) observed at higher values of ϵ .
- The ART implementation almost matched the from-scratch FGSM outputs, validating both implementations.

Adversarial Detection — PGD and BIM

Detection Model Setup

- **Detector Architecture:** ResNet-34 with binary output head (clean = 0, adversarial = 1)
- **Training Data:** Mixed clean + adversarial images (50/50 split)
- **Adversarial Generation:** IBM ART — `ProjectedGradientDescent` and `BasicIterativeMethod`
- **Loss:** Binary Cross-Entropy
- **Optimizer:** Adam

PGD-based Adversarial Detection

Epoch	Train Loss	Val Acc (%)
1	0.0935	99.94
2	0.0029	100.00
3	0.0005	100.00
4	0.0001	100.00
5	0.0000	100.00
6	0.0000	100.00
7	0.0000	100.00
8	0.0000	100.00
9	0.0000	100.00
10	0.0000	100.00

PGD adversarial images

BIM-based Adversarial Detection

Epoch	Train Loss	Val Acc (%)
1	0.0951	98.72
2	0.0066	99.88
3	0.0022	99.94
4	0.0018	99.97
5	0.0008	99.97
6	0.0015	99.91
7	0.0035	100.00
8	0.0001	100.00
9	0.0000	100.00
10	0.0000	100.00

BIM adversarial images

Detection Performance Comparison

Attack Type	Attack Method	Detection Acc (%)
Clean vs PGD	IBM ART PGD	100
Clean vs BIM	IBM ART BIM	100

Key Observations — Adversarial Detection

- BIM takes more time to converge as compared to PGD
- PGD is a stronger, iterative attack; its adversarial images exhibit more structured perturbations that the detector can exploit as distinguishing features.
- BIM (a variant of PGD without projection) also introduces detectable statistical artifacts, but detection may vary with step size and iteration count.
- The ResNet-34 detector learned to distinguish clean vs adversarial images by picking up high-frequency noise patterns introduced by gradient-based attacks.

WandB Visualisations

The following results were logged to WandB:

- Q1: Train-val loss & accuracy curves for all 9 LoRA experiments + baseline, class-wise test accuracy histogram, gradient update norms on LoRA weights.
- Q2: 10 sample images (per category) showing:
 - Original clean image
 - FGSM adversarial (from scratch)
 - FGSM adversarial (IBM ART)
 - PGD adversarial (IBM ART)
 - BIM adversarial (IBM ART)

Links

- **GitHub (Assignment 5 branch):** https://github.com/arpitadeshmukh/MLOps-Arpita_Abhijit_Deshmukh-B23CM1007/tree/Assignment5
- **WandB Link Q1:** <https://wandb.ai/b23cm1007-indian-institute-of-technology-jodhpur/MLOPS%20Assignment-5/workspace?nw=nwuserb23cm1007>

- **WandB Link Q2:** <https://wandb.ai/b23cm1007-indian-institute-of-technology-jodhpur/MLOPS-Assi-runs/04wc8aso?nw=nwuserb23cm1007>
- **HuggingFace (Best LoRA Model):** <https://huggingface.co/arpita2desh/Vit-Lora-CIFAR100-MLOPS>
- **HuggingFace (Adversarial Attack):** <https://huggingface.co/arpita2desh/Adversarial-cifar10-MLOP>